

CS6008 - Network Security

Assignment 1: Building a Secure Chat Application

P Saatvika 23B0935 S Joel Tenny 23B1002

September 4, 2026

Contents

1	Phase 1: Baseline Chat Application (No Security)	2
1.1	Protocol Description	2
1.2	Required Verification	2
2	Phase 2: Client-Server Confidentiality via Diffie-Hellman	4
2.1	Key Derivation	4
2.2	Required Verification	4
2.3	Man-in-the-Middle Attack (Phase 2)	5
3	Phase 3: Server Authentication via PKI	6
3.1	PKI Setup Commands	6
3.2	Legitimate Flow	6
3.3	Defeating the MITM Proxy	7
3.4	Defeating a Stolen Certificate (Proof of Possession)	7
4	Phase 4: End-to-End Encryption Between Clients	9
4.1	Implementation	9
4.2	Required Verification	9
5	Phase 5: Forward Secrecy	11
5.1	Collision Avoidance Strategy	11
5.2	Required Verification	11
5.3	The Value of Forward Secrecy	12

1 Phase 1: Baseline Chat Application (No Security)

Network setup: Server 192.168.1.10; Client 1 (Alice) 192.168.1.11; Client 2 (Bob) 192.168.1.12.

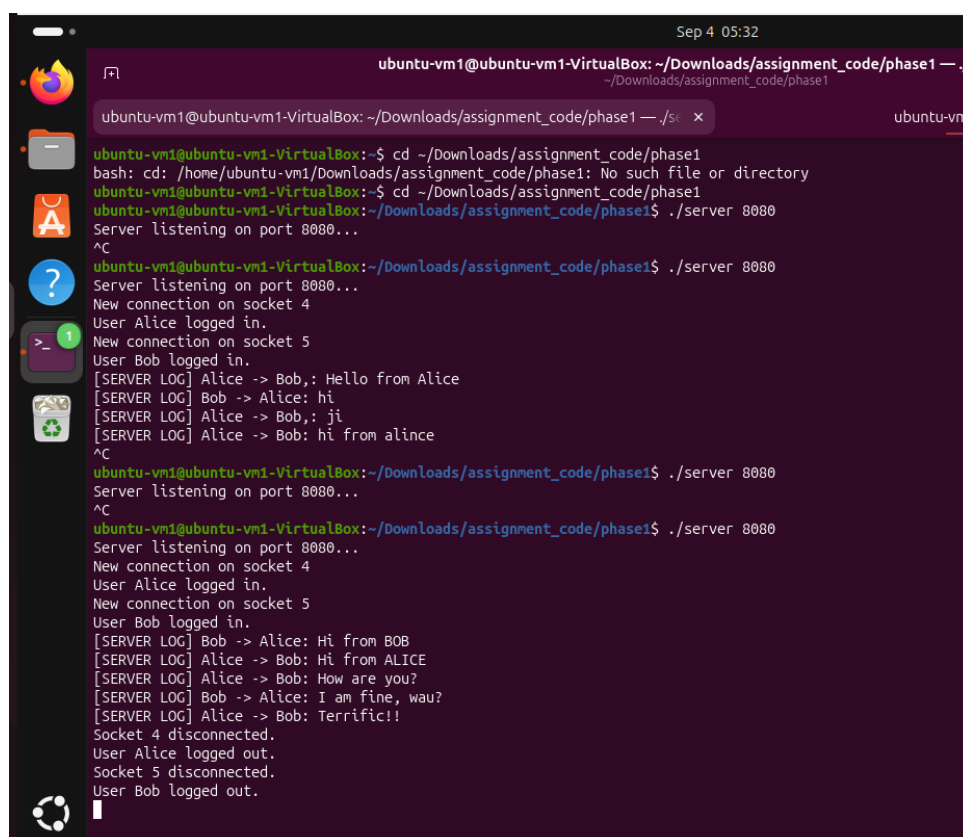
1.1 Protocol Description

The application uses newline-delimited strings for message framing over TCP. A message is considered complete when a `\n` character is received by the socket buffer.

To route messages, the server maintains an internal mapping of `username` $\rightarrow$ `socket`. When a client connects and sends the `LOGIN <username>` command, the server registers them in this mapping. When the server receives a message prefixed with `@<target>`, it parses the target username, looks up their socket in the mapping, and forwards the message exactly as-is. If more than two clients were to connect, this mapping mechanism naturally scales to route messages dynamically to any connected user.

1.2 Required Verification

The server successfully routes messages between Alice and Bob without any encryption. As shown in the server logs below, the server is able to read the exact plaintext of every message passing through it.



```
ubuntu-vm1@ubuntu-vm1-VirtualBox: ~/Downloads/assignment_code/phase1
ubuntu-vm1@ubuntu-vm1-VirtualBox: ~/Downloads/assignment_code/phase1
ubuntu-vm1@ubuntu-vm1-VirtualBox: ~/Downloads/assignment_code/phase1$ ./server 8080
Server listening on port 8080...
^C
ubuntu-vm1@ubuntu-vm1-VirtualBox: ~/Downloads/assignment_code/phase1$ ./server 8080
Server listening on port 8080...
New connection on socket 4
User Alice logged in.
New connection on socket 5
User Bob logged in.
[SERVER LOG] Alice -> Bob,: Hello from Alice
[SERVER LOG] Bob -> Alice: hi
[SERVER LOG] Alice -> Bob,: ji
[SERVER LOG] Alice -> Bob: hi from alicne
^C
ubuntu-vm1@ubuntu-vm1-VirtualBox: ~/Downloads/assignment_code/phase1$ ./server 8080
Server listening on port 8080...
^C
ubuntu-vm1@ubuntu-vm1-VirtualBox: ~/Downloads/assignment_code/phase1$ ./server 8080
Server listening on port 8080...
New connection on socket 4
User Alice logged in.
New connection on socket 5
User Bob logged in.
[SERVER LOG] Bob -> Alice: Hi from BOB
[SERVER LOG] Alice -> Bob: Hi from ALICE
[SERVER LOG] Alice -> Bob: How are you?
[SERVER LOG] Bob -> Alice: I am fine, wau?
[SERVER LOG] Alice -> Bob: Terrific!!
Socket 4 disconnected.
User Alice logged out.
Socket 5 disconnected.
User Bob logged out.
```

Figure 1: Phase 1 Server Logs showing intercepted plaintext

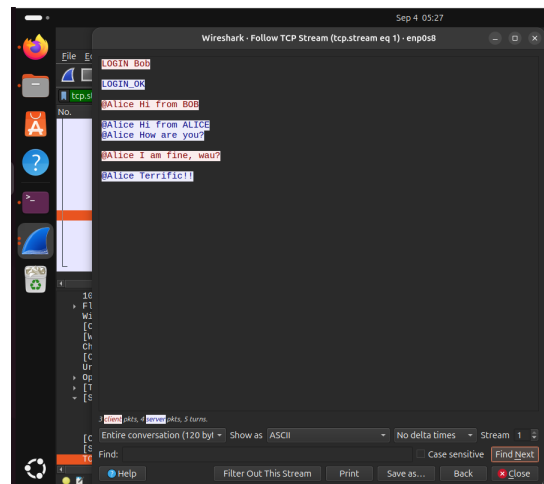
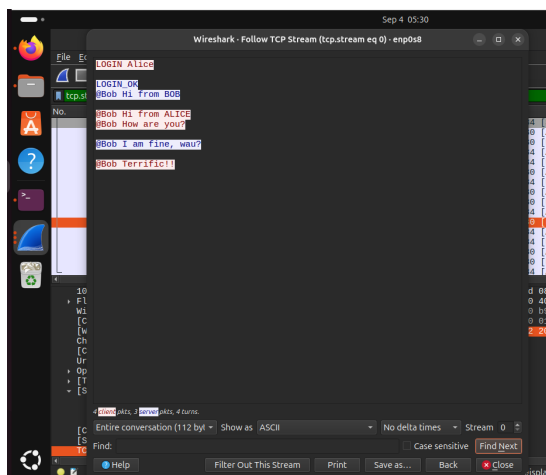


Figure 2: Phase 1 Client Terminals (Alice and Bob)

2 Phase 2: Client-Server Confidentiality via Diffie-Hellman

Network setup: Server 192.168.1.10; Client 1 (Alice) 192.168.1.11; Client 2 (Bob) 192.168.1.12; Mallory 192.168.1.13 for the MITM demonstration.

2.1 Key Derivation

Why hash the raw Diffie-Hellman shared secret?

The raw Diffie-Hellman shared secret is a mathematically structured integer (a group element in the MODP group), not a uniformly random bit string. Using it directly as an AES key exposes the encryption to algebraic weaknesses and bias. Passing it through a cryptographic hash function like SHA-256 acts as a robust Key Derivation Function (KDF), uniformly distributing the entropy and decoupling the mathematical structure from the symmetric encryption key.

2.2 Required Verification

Both ends of the Diffie-Hellman exchange successfully derive the identical SHA-256 hashed fingerprint.

```
ubuntu-vm1@ubuntu-vm1-VirtualBox: ~/Downloads/assignment_code/phase2
User Bob logged out.
^C
ubuntu-vm1@ubuntu-vm1-VirtualBox:~/Downloads/assignment_code/phase1$ cd ./
./././
ubuntu-vm1@ubuntu-vm1-VirtualBox:~/Downloads/assignment_code/phase2$ ./server 8080
Phase 2 Server listening on port 8080...
New connection on socket 4
[Socket 4] Shared Secret Fingerprint: 74093b0b48aef052559fddc3b20f5e03eef
f9642214cd0036437c98e0eb7739
New connection on socket 5
[Socket 5] Shared Secret Fingerprint: 41a50963ac738c047bdf9dffa8bd62770eaa
be9090c6a90e1c30465e055007
User Alice logged in.
[SERVER LOG] Bob -> Alice: Hi from bob
[SERVER LOG] Alice -> Bob: hello!
[SERVER LOG] Alice -> Bob: /tamper I am a hacker
[SERVER LOG] Forwarded intentionally TAMPERED ciphertext to Bob!
^C
ubuntu-vm1@ubuntu-vm1-VirtualBox:~/Downloads/assignment_code/phase2$ ./server 8080
Phase 2 Server listening on port 8080...
New connection on socket 4
[Socket 4] Shared Secret Fingerprint: fcf82ea2bf1f7cbf1f473591e47fc21d910
3dd7c7f5fdd28d5db08f5e24ba3b
User Alice logged in.
New connection on socket 5
[Socket 5] Shared Secret Fingerprint: 52c593102a57c8119dc56617beab3e0c0e4a
05cf151f32a678a7b73c6a9d5ce9
User Bob logged in.
```

```
File Machine View Input Devices Help
ubuntu-vm1@ubuntu-vm1-VirtualBox: ~/Downloads/assignment_code/phase2
Disconnected from server.
^C
ubuntu-vm1@ubuntu-vm1-VirtualBox:~/Downloads/assignment_code/phase1$ ./client 192.168.1.10 8080
Enter your username: Alice
Logged in successfully.
>
[Bob]: Hi from BOB
> @Bob Hi from ALICE
> How are you?
>
[Bob]: I am fine, wau?
> Terrific!!
^C
ubuntu-vm1@ubuntu-vm1-VirtualBox:~/Downloads/assignment_code/phase1$ cd ./
./././
ubuntu-vm1@ubuntu-vm1-VirtualBox:~/Downloads/assignment_code/phase2$ ./client 192.168.1.10 8080
[DH Handshake Complete] AES Key Fingerprint: 74093b0b48aef052559fddc3b20f
5e03eef9642214cd0036437c98e0eb7739
Enter your username: Alice
Logged in successfully.
>
[Bob]: Hi from bob
> Bob hello!
No chat partner selected. Use /chat <username> or @username <message>
> @Bob hello!
> @Bob /tamper I am a hacker
>
Disconnected from server.
^C
ubuntu-vm1@ubuntu-vm1-VirtualBox:~/Downloads/assignment_code/phase2$ ./client 192.168.1.10 8080
[DH Handshake Complete] AES Key Fingerprint: fcf82ea2bf1f7cbf1f473591e47fc21d9103dd7c7f5fdd28d5db08f5e24ba3b
Enter your username: Alice
Logged in successfully.
```

Figure 3: Phase 2 Identical Fingerprints between Client 1 and Server

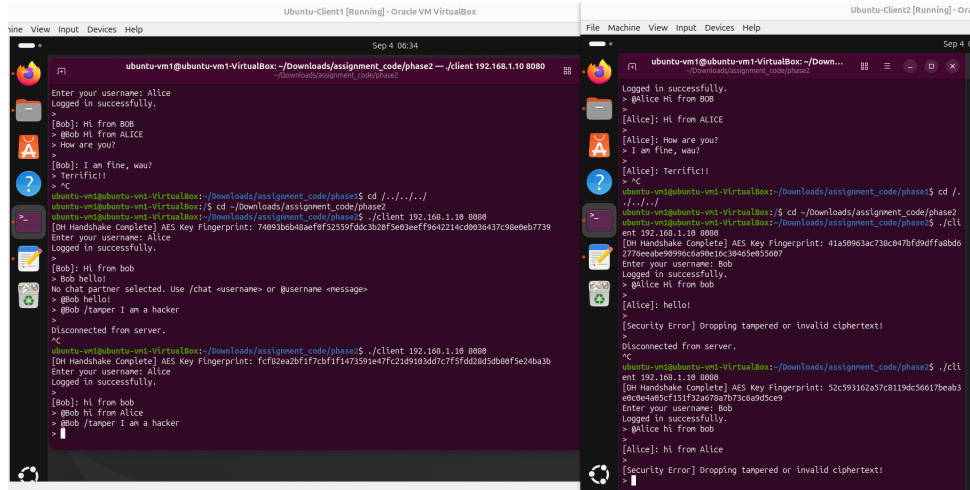


Figure 4: Phase 2 Encrypted Chat flowing correctly

2.3 Man-in-the-Middle Attack (Phase 2)

Because Diffie-Hellman provides confidentiality but no authentication, a Man-in-the-Middle (MITM) proxy can intercept the connection, perform separate DH handshakes with both the client and the server, and decrypt all traffic.

Observable Evidence vs User Perception:

If a victim were actively verifying the DH fingerprints out-of-band (e.g., over a phone call with the server administrator), they would notice the MITM attack because their locally computed fingerprint would not match the real server's fingerprint. However, an ordinary user would be completely oblivious because the chat application functions perfectly; the traffic remains encrypted on the wire (preventing passive eavesdropping by Wireshark), and no warnings are generated.

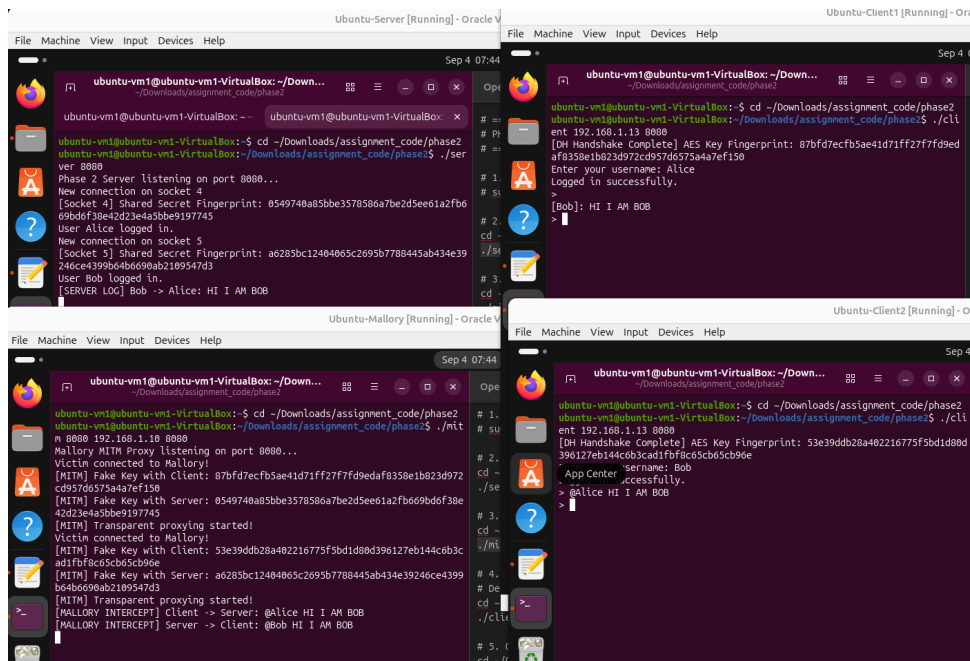


Figure 5: Phase 2 MITM Attack Successfully Capturing Plaintext

3 Phase 3: Server Authentication via PKI

Network setup: Server 192.168.1.10; Client 1 (Alice) 192.168.1.11; Client 2 (Bob) 192.168.1.12; Mallory 192.168.1.13 for the MITM demonstration.

3.1 PKI Setup Commands

The following OpenSSL commands were used to create the Certificate Authority and issue the server's certificate:

```
# 1. Generate CA Private Key and Self-Signed Root Certificate
openssl genpkey -algorithm RSA -out ca.key -pkeyopt
    rsa_keygen_bits:2048
openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 -out
    ca.crt -subj "/C=US/O=MyCA/CN=My Root CA"

# 2. Generate Server Private Key and Certificate Signing Request
openssl genpkey -algorithm RSA -out server.key -pkeyopt
    rsa_keygen_bits:2048
openssl req -new -key server.key -out server.csr -subj "/C=US/O=
    MyServer/CN=192.168.1.10"

# 3. Sign the Server CSR using the CA to produce the Server
    Certificate
openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -
    CAcreateserial -out server.crt -days 365 -sha256 -extfile <(
    printf "basicConstraints=CA:FALSE\nsubjectAltName=IP
    :192.168.1.10")
```

3.2 Legitimate Flow

The client connects, successfully validates the signature and CN of the certificate against the trusted Root CA, verifies the Proof-of-Possession signature, and completes the DH handshake.

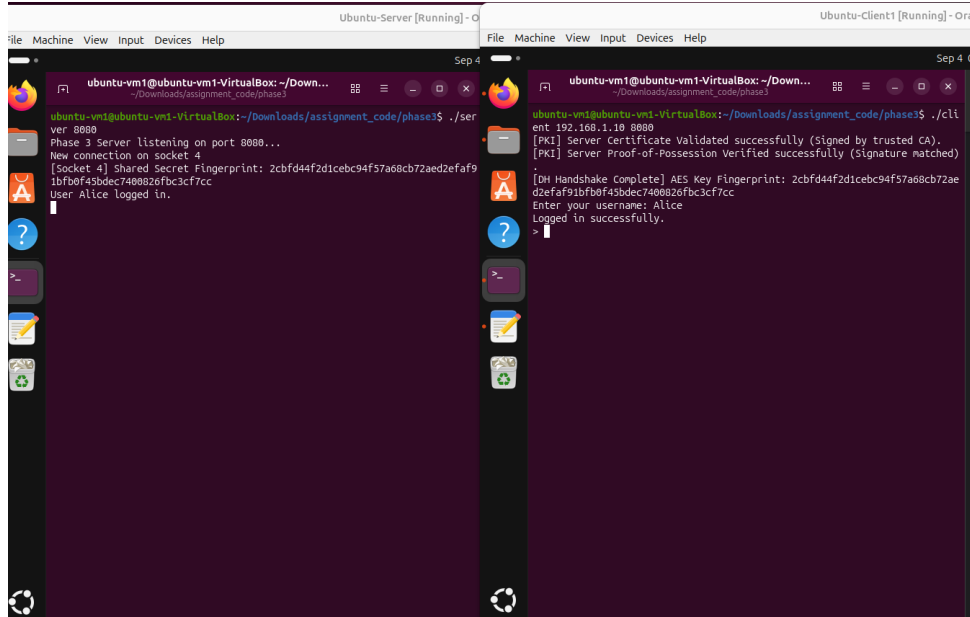


Figure 6: Phase 3 Successful PKI Validation

3.3 Defeating the MITM Proxy

When the Phase 2 MITM proxy attempts the same attack, it must present a certificate. Because the proxy lacks the CA's private key, it generates a fake certificate. The client's validation logic immediately rejects this unverified signature and drops the connection, completely defeating the attack.

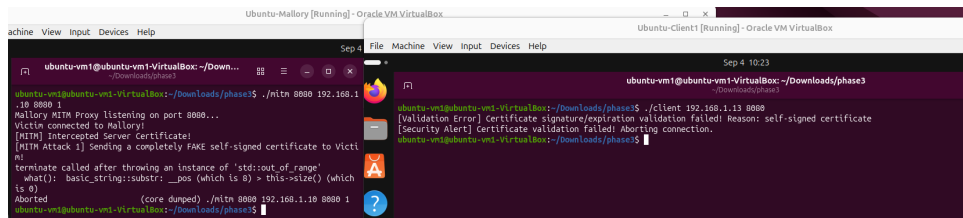


Figure 7: Phase 3 Defeating the MITM Attack (Certificate Verification Failed)

3.4 Defeating a Stolen Certificate (Proof of Possession)

To prove possession of the private key corresponding to the certificate, the server signs its ephemeral Diffie-Hellman public key during the handshake. The client extracts the RSA public key from the validated certificate and verifies this signature. If an attacker steals the certificate file but not the private key, they cannot produce a valid signature over a fresh DH key. The client immediately drops the connection.

Proof-of-Possession Implementation (Server Side):

```
// Server generates DH Key and signs it with its RSA Private Key
DHKeyExchange dh;
string server_pub = dh.get_public_key_hex();
string signature = sign_data(server_pub, private_key);

// Server sends DH Key + Signature to Client
string reply = "DH_ACK_" + server_pub + "_" + string_to_hex(signature) + "\n";
send(client_sock, reply.c_str(), reply.length(), 0);
```

```
ubuntu-vm1@ubuntu-vm1-VirtualBox: ~/Downloads
./mitm 8080 192.168.1.13 8080
[MITM] MITM Proxy listening on port 8080...
Victim connected to Mallory!
[MITM] Intercepted Server Certificate!
[MITM Attack 2] Forwarding the STOLEN Real Server Certificate to Victim!
[MITM] Attempting Proof-of-Possession Bypass!
[MITM] Sending Mallory's Fake DH Public Key but attaching the Real Server's Signature...
[MITM] Fake Key with Client: 0d9ee05083126d21de41a8194744ee031b115afb2327b9ee0ffcc2217202
[MITM] Fake Key with Server: 3395182701d25f34281d9797e77d65dd7b1906d0a02938b07413699135f000f
[MITM] Transparent proxying started!

ubuntu-vm1@ubuntu-vm1-VirtualBox: ~/Downloads/phase3
./client 192.168.1.13 8080
[PKI] Server Certificate Validated successfully (Signed by trusted CA).
[Security Alert] Proof-of-Possession signature verification failed! Server does not possess private key. Aborting!
```

Figure 8: Phase 3 Defeating a Stolen Certificate via Proof-of-Possession

4 Phase 4: End-to-End Encryption Between Clients

Network setup: Server 192.168.1.10; Client 1 (Alice) 192.168.1.11; Client 2 (Bob) 192.168.1.12.

4.1 Implementation

Phase 4 implements a secondary, inner layer of encryption. When the `/e2e <username>` command is issued, the client initiates a secondary DH Key Exchange explicitly with the partner. The server blindly routes the `_E2E_INIT__` and `_E2E_ACK__` tags. Once the inner E2E AES-GCM key is established, all chat messages are double-encrypted.

Double-Encryption Implementation (Client Side):

```
// 1. Inner Encryption (Client-to-Client E2E Key)
string inner_enc = encrypt_aes_gcm(msg, e2e_key);

// 2. Wire Tagging for Server Routing
string packet = "@" + partner + "_E2E_MSG_" + inner_enc;

// 3. Outer Encryption (Client-to-Server Link Key) inside send_packet()
string outer_enc = encrypt_aes_gcm(packet, server_key);
send(sock, outer_enc.c_str(), outer_enc.length(), 0);
```

4.2 Required Verification

The screenshots demonstrate that both clients derive the exact same E2E Key Fingerprint independently, without the server's involvement. Furthermore, the Server's routing logs prove that it only sees opaque, unreadable ciphertext for the chat content.

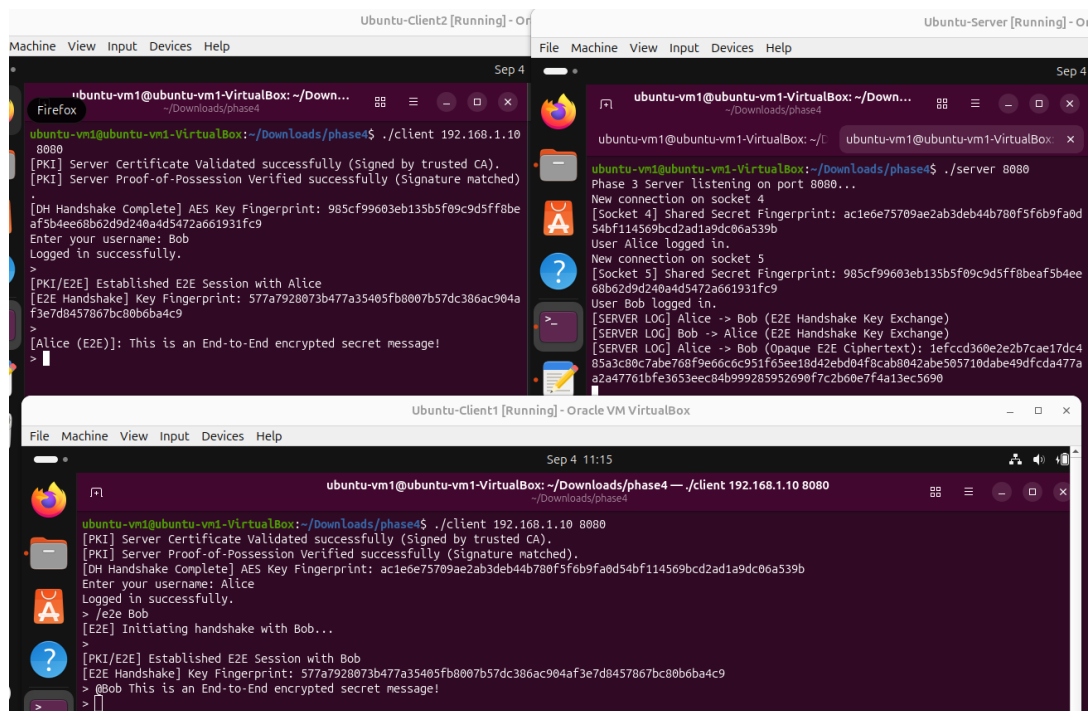


Figure 9: Phase 4 E2E Fingerprints and Server Blindness

5 Phase 5: Forward Secrecy

Network setup: Server 192.168.1.10; Client 1 (Alice) 192.168.1.11; Client 2 (Bob) 192.168.1.12.

5.1 Collision Avoidance Strategy

To handle the case where both clients might attempt to trigger a key renegotiation simultaneously, the design utilizes **Lexicographical Leader Election**. Instead of both clients running timers, the clients compare their usernames alphabetically. Only the client whose username is lexicographically smaller (e.g., Alice < Bob) acts as the "Leader" and initiates the 60-second rotation timer. The other client acts passively. This completely eliminates the possibility of simultaneous collisions and guarantees a deterministic, agreeable state.

Collision-Avoidance Logic (Client Side):

```
// Phase 5 Collision Avoidance: Lexicographical Leader Election  
// Only the client whose username is alphabetically smaller starts the background timer.  
// This mathematically prevents both clients from rotating at the same time.  
if (my_username < partner_username) {  
    thread t(rotation_thread, partner_username, sock);  
    t.detach();  
}
```

5.2 Required Verification

The background timer successfully negotiates a genuinely new E2E key every 60 seconds. In-flight messages are gracefully handled by retaining the `previous_key` during the rotation window, ensuring zero disruption to the ongoing chat.

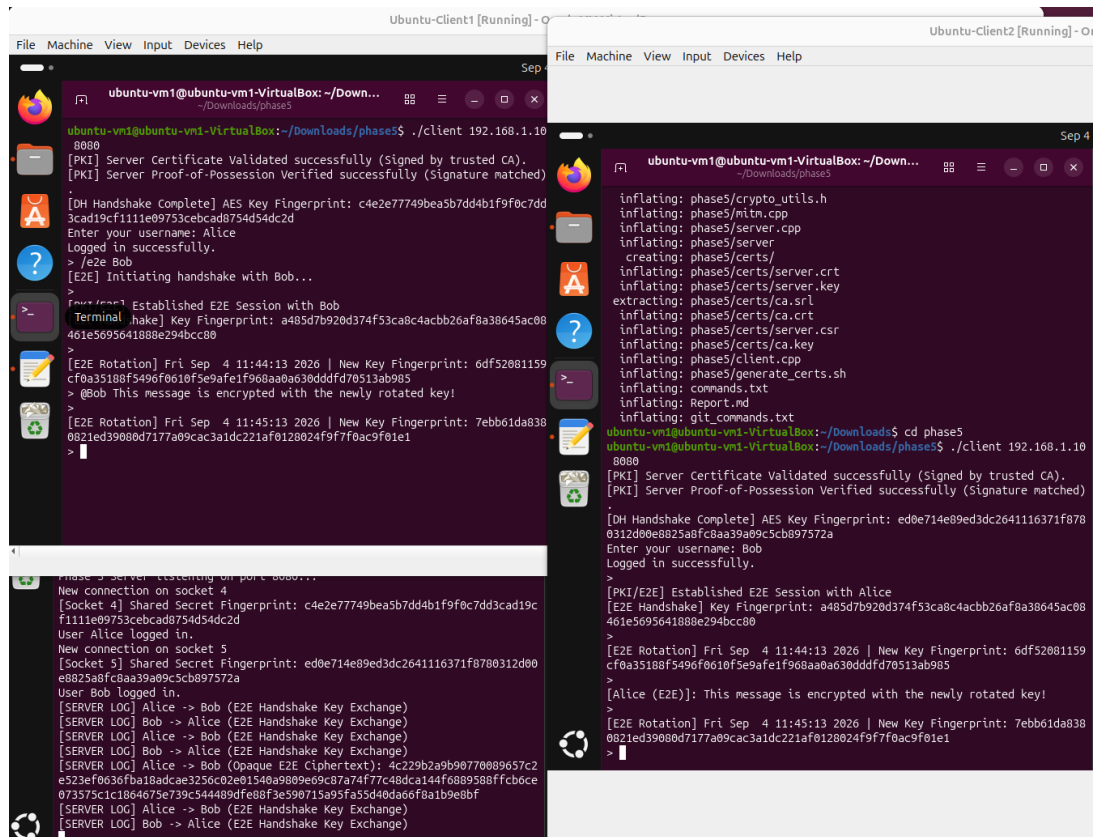


Figure 11: Phase 5 Automatic Key Rotation (Forward Secrecy)

5.3 The Value of Forward Secrecy

What does forward secrecy buy you that Phase 4 alone did not?

In Phase 4, a single long-lived session key protects the entire conversation. If a nation-state or passive attacker records all the encrypted traffic and later compromises one of the client's machines to steal that single session key, they can retroactively decrypt the entire past conversation.

Forward Secrecy prevents this. By continuously rotating keys and irreversibly discarding the old ones, a compromise of the current key only exposes the messages sent during that specific 60-second window. All historical messages remain perfectly secure because the keys used to encrypt them have been permanently destroyed.